

SkyPoints Airline Loyalty Platform

Senior Azure Data Engineering Architecture, Execution & Interview Master Guide

Target Scale: Multi-Billion Records/Day (~2-5 TB)	Storage: ADLS Gen2 (Hierarchical Namespace)	Compute: Azure Databricks (PySpark / Delta)
Orchestration: Azure Data Factory (ADF)	Serving: Snowflake DDL / Delta Country Marts	Status: 100% Tests Passing & Deployed Live

1. Medallion Architecture & End-to-End Flow

SkyPoints processes millions to billions of member profiles and mileage redemption transactions daily. To guarantee high throughput, data integrity, and compliance across countries, the solution implements the **Medallion Architecture (Bronze → Silver → Gold)** on Azure.

Layer	Container / Storage	Key Transformations & Business Logic	Output Artifacts
BRONZE (Landing / Raw)	ADLS Gen2 landing/ bronze/	- Raw ingest of pipe-delimited profile feed and nested JSON. - Immutable audit trail preserving raw payload & line numbers.	RAW_MEMBER_PROFILES RAW_REDEMPTION_FEED
SILVER (Staging / Cleansed)	ADLS Gen2 silver/ (Delta Lake)	- Strip Header (IH) and parse Detail (ID) records. Age calculation from DOB & Stale_Member flag (>90 days). - Recover lost leading zero on DOB (3051985 → 03051985). - Quarantine dead-letter routing for invalid records.	STG_MEMBER_PROFILES quarantine_members
GOLD (Marts / Serving)	ADLS Gen2 gold/ (Delta Lake)	Latest Record Wins windowing (ROW_NUMBER()). - Country routing (table_usa, table_ind, table_can...). - Explode nested JSON redemptions into flat rows. Member 360 aggregation join via broadcast hash join.	table_can, table_ind fact_redemptions member_redemptions_360

2. Step-by-Step Pipeline Execution Guide

The platform is 100% portable and can be executed across three execution modes: locally for rapid development & automated tests, in Azure Databricks for distributed compute, or orchestrated via Azure Data Factory.

Execution Environment	Exact Commands to Run	Expected Verification Results
1. Local Execution (Full ETL Pipeline)	python -m src.pipeline	Runs full Medallion pipeline. Parses sample flat file & JSON, calculates Age & Stale flag, performs latest-wins deduplication, routes to country JSON files in data/output/.
2. Automated Tests (100% Pass Rate)	python -m pytest -v	Executes 8 automated unit tests verifying header filtering, JSON explosion, DOB zero-padding recovery, age logic, flight-date sequence gates, and country routing.

3. Azure Data Factory (Cloud Orchestration)	az datafactory pipeline create-run --factory-name adf-skypoints-loyalty --name pipeline_skypoints_orchestrator	Triggers GetMetadata gatekeeper validation in ADLS Gen2 landing container. Verifies files exist, validates schemas, and orchestrates downstream processing (Run ID: eba929dc...).
4. Azure Databricks (Serverless Compute)	In Databricks Workspace: Import notebooks/skypoints_databricks_pipeline.py Click Run All	Streams raw files from ADLS Gen2, builds PySpark DataFrames, saves Delta Lake tables (table_usa, table_ind, table_can, fact_redemptions) in Unity Catalog.

3. Detailed Component Deep-Dive

3.1 Ingestion & The Delimiter Alignment Trap

The Problem: The member flat file uses mixed records (|H|... header, |D|... details). Because lines begin with a leading pipe (e.g. |D|Elena|223457|...), a standard split produces an empty token at index 0.

The Solution: We apply `regexp_replace(col('value'), '^\\|', '')` before splitting. This normalizes index 0 as record type (H or D), index 1 as Member_Name, and index 2 as Member_Id.

3.2 Recovering Dropped Leading Zeroes on DOB

The Problem: In the assessment's intermediate table, DOB was displayed as 3051985 instead of 03051985. Upstream systems cast the string to an integer, stripping the leading zero for months January through September. Standard date parsers throw exceptions on 7-character dates.

The Solution: We use `LPAD(raw_dob, 8, '0')` to guarantee an 8-character string, then coalesce `MMddyyyy` and `ddMMyyyy` formats.

3.3 Staging Derived Columns (Deliverable 2)

Age Calculation: `floor(datediff(current_date, date_of_birth) / 365.25)` handles leap-year drift accurately.

Stale Member Flag: `CASE WHEN last_flight_date IS NULL OR datediff(current_date, last_flight_date) > 90 THEN 'Y' ELSE 'N' END`. Members who have never flown or whose last flight was \$>90\$ days ago are flagged as Stale.

3.4 Country Routing & 'Latest Record Wins' Windowing (Deliverable 3)

The Relocation Problem: When a member moves across national borders, they must not have active records in multiple country marts. In the sample data, Elena had an initial flight in the USA (2012) and a subsequent flight in Canada (2024).

The Solution: We partition by member_id and sort descending by last_flight_date, enrollment_date:

`ROW_NUMBER() OVER (PARTITION BY member_id ORDER BY last_flight_date DESC NULLS LAST, enrollment_date DESC) AS rank`.

Filtering for rank = 1 places Elena exclusively in table_can and deactivates her previous USA record.

3.5 Semi-Structured JSON Redemption Flattening & Member 360 (Deliverable 4)

JSON Explosion: The partner feed provides a top-level member_id and feed_date, with a nested array redemptions: `[{txn_id, txn_date, partner, miles_redeemed, status}]`. Using PySpark `explode('redemptions')` unpacks each transaction into a discrete row in fact_redemptions.

Broadcast Join: Redemption metrics are aggregated by member_id (total miles, completed vs. pending miles) and joined back to the member profile via a **Broadcast Hash Join**, avoiding expensive cluster-wide shuffles across billions of member rows.

3.6 Data Quality Quarantine Dead-Letter Pattern (Deliverable 5)

Rather than halting multi-hour batch runs when a malformed record appears, records failing validation (e.g., missing member_id, unparseable dates, or flight dates preceding enrollment) are routed to an append-only Delta sink (quarantine_members) with explicit rejection reasons (e.g. *'Missing mandatory field member_id'*).

4. Multi-Billion Scale Engineering Strategy

Challenge	Senior Architectural Strategy	Technical Implementation
File Discovery Overhead	Avoid full directory scans over millions of daily files.	Databricks Auto Loader (cloudFiles): Uses Azure Event Grid blob notifications to detect newly arrived files without directory crawl.
Data Skew & Country Routing	Avoid doing 50 table scan loops across billions of rows.	Dynamic Partition Overwrite: Write once with partitionBy('country'). Catalyst optimizer routes data to physical partition directories in a single distributed pass.
The Small-File Problem	Frequent streaming/batch writes create millions of tiny Parquet files.	Compaction & Z-Ordering: Execute OPTIMIZE members_by_country ZORDER BY (member_id) to merge into 1GB files and co-locate rows on disk.
Shuffle Bottlenecks on Join	Joining billions of profiles to redemptions causes network exhaustion.	Broadcast Hash Join: Broadcast the aggregated redemptions table (megabytes) across worker nodes to avoid shuffling the member profile table (terabytes).

5. One-Liner Technical Interview Cheat Sheet

Crisp, high-impact responses to technical questions for your architectural interview:

Q1: What is the primary key of the member profile table?

→ Member ID is the primary entity key; while the assessment spec marked Member Name as key, in real-world loyalty systems names have collisions, requiring a unique business surrogate key.

Q2: How did you fix the missing leading zero in the DOB column?

→ We used LPAD(raw_dob, 8, '0') to recover dropped leading zeroes (converting '3051985' to '03051985') before casting to date.

Q3: What logic determines if a member is flagged as Stale?

→ A member is marked Stale ('Y') if their last flight date is NULL (never flown) or if datediff(current_date, last_flight_date) > 90 days.

Q4: How does the pipeline handle members moving to another country?

→ We apply a window ranking ROW_NUMBER() partitioned by member_id and ordered by last_flight_date DESC; rank 1 routes to the new country table, and the prior country record is deactivated.

Q5: Why did you use ADLS Gen2 with Hierarchical Namespace (HNS)?

→ HNS enables true atomic directory renames and POSIX access control, essential for Delta Lake ACID transactions and petabyte-scale throughput.

Q6: How do you parse the semi-structured JSON redemption feed in PySpark?

→ We use PySpark's explode() function on the nested 'redemptions' array to unpack each transaction into a discrete, queryable row.

Q7: Why use a Broadcast Hash Join when joining Redemptions to Member Profiles?

→ The aggregated redemptions table is orders of magnitude smaller than the multi-billion member table; broadcasting avoids cluster-wide network shuffles.

Q8: Why not use standard Python/Pandas for this pipeline?

→ Billions of daily records represent 2-5 TB uncompressed; Pandas is single-threaded and memory-bound (OOM), whereas PySpark distributes computation across worker nodes.

Q9: What is the purpose of GetMetadata activities in Azure Data Factory?

→ They serve as cost-optimization gatekeepers, validating that daily landing files exist and are non-empty before spinning up expensive compute clusters.

Q10: What happens when a malformed record enters the pipeline?

→ It routes to an append-only dead-letter quarantine table (quarantine_members) with failure reason tags, preventing job abortion.

Q11: How do you prevent data skew when writing to country target tables?

→ We use Delta Dynamic Partition Overwrites partitioned by country, allowing Spark to write into target partition directories in a single distributed pass.

Q12: How do you mitigate the 'small-file problem' in Delta Lake?

→ We run OPTIMIZE members_by_country ZORDER BY (member_id) to compact small files into 1GB Parquet blocks and co-locate records.

Q13: Why bypass spark.conf.set on Databricks Serverless compute?

→ Serverless enforces strict Unity Catalog security and blocks cluster-level Hadoop filesystem modifications; using the Azure Storage SDK provides native cloud stream access.

Q14: How is incremental processing handled at billions scale?

→ We use Databricks Auto Loader (cloudFiles) for event-driven file ingestion and Delta Lake MERGE INTO for atomic change data capture (CDC).

Q15: How did you handle the schema discrepancy regarding the Post Code column?

→ The markdown spec listed Post Code at position 9, but the sample flat file omitted it; our parser uses flexible token length mapping to prevent off-by-one errors.